

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования

**“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”**

Факультет прикладной информатики (фПИИ)

Направление подготовки (специальность) – 45.03.04 (Языковые модели и
Искусственный Интеллект)

Проектирование и реализация баз данных

Лабораторная работа № 3-4 часть 2

Выполнил студент: Попов Глеб Ильич Группа № К3260

Преподаватель: Харлуниин Александр Александрович

г. Санкт-Петербург 2026

Содержание

Содержание.....	2
1. Цель работы.....	3
2. Используемые технологии.....	3
3. Ход выполнения работы и реализация заданий.....	3
3.1. Пакетная обработка (Batch Processing) и управление очередью.....	3
3.2. Агрегация, препроцессинг и очистка данных (Data Cleaning).....	3
3.3. Извлечение скрытых зависимостей (Дополнительное задание №1).....	4
3.4. Программная защита от галлюцинаций LLM (Дополнительное задание №2).....	4
4. Листинг исходного кода.....	5
5. Результаты выполнения скрипта.....	12
6. Вывод.....	15

1. Цель работы

Разработать отказоустойчивый конвейер обогащения данных (Data Enrichment) для базы данных хоккейной статистики NHL с использованием большой языковой модели (LLM). Интегрировать GigaChat API с локальной СУБД MongoDB для генерации аналитических сводок, тегов и выявления скрытых паттернов матча. Реализовать защиту пайплайна от галлюцинаций нейросети с помощью агрегации данных, очистки контекста (Data Cleaning) и жесткой программной валидации структуры (схемы) ответа.

2. Используемые технологии

- **СУБД:** MongoDB Community Server (База данных: nhl_db, коллекции: games, teams).
- **Языковая модель:** GigaChat API.
- **Язык программирования:** Python 3.11.
- **Ключевые библиотеки:** pymongo (драйвер БД), gigachat (API клиент), json (сериализация), re (регулярные выражения), python-dotenv (безопасное управление ключами).

3. Ход выполнения работы и реализация заданий

3.1. Пакетная обработка (Batch Processing) и управление очередью

Для предотвращения перегрузки API и защиты от прерываний реализована пакетная обработка документов (по 15 матчей за запуск). Очередь формируется динамически с помощью агрегационного фильтра {"\$match": {"ai_processed": {"\$ne": True}}}. Это позволяет скрипту игнорировать успешно обработанные документы и автоматически отправлять на повторный анализ те матчи, при обработке которых ранее произошла ошибка или сработала блокировка валидатора.

3.2. Агрегация, препроцессинг и очистка данных (Data Cleaning)

Для исключения галлюцинаций, связанных с незнанием языковой моделью идентификаторов команд (ObjectId), применен оператор \$lookup. Данные о командах подтягиваются из коллекции teams. Чтобы избежать транслитерации и перевода названий команд моделью (например, «Питтсбург Пингвинз»), перед отправкой промпта проводится очистка: из

подтянутых документов извлекаются строгие оригинальные названия на английском языке (`home_team_name`, `away_team_name`), а все технические связи и массивы удаляются. Также в конвейер интегрирована функция предварительного подсчета голов `build_goal_summary`, которая снабжает LLM готовой математикой для безошибочного определения MVP матча.

3.3. Извлечение скрытых зависимостей (Дополнительное задание №1)

Помимо базовой генерации описания (`ai_tldr`) и тегирования (`ai_tags`), функционал обогащения расширен аналитическими задачами. LLM анализирует ход матча и генерирует блок `ai_hidden_insights`, содержащий:

- `game_intensity` - оценка напряженности матча (Low / Medium / High).
- `mvp_candidate` - кандидат на звание лучшего игрока матча (выбирается строго из авторов забитых шайб).
- `turning_point` - аналитическое определение переломного момента игры.

3.4. Программная защита от галлюцинаций LLM (Дополнительное задание №2)

Реализован многоуровневый механизм защиты базы данных от некорректных или "креативных" ответов нейросети:

1. **Изоляция полезной нагрузки:** Функция `extract_and_parse_json` использует регулярное выражение `re.search(r'\{.*\}', text_response, re.DOTALL)` для извлечения чистого JSON-ядра, отсекая Markdown-разметку.
2. **Жесткая валидация схемы (Whitelist Validation):** Написан строгий валидатор `validate_schema`. Он проверяет количество тегов (от 2 до 5) и сверяет их с разрешенным "белым списком" (например: "Разгром", "Камбэк", "Волевая победа"). Любая попытка модели сгенерировать неформатный тег приводит к программной блокировке записи.
3. **Логирование ошибок в БД:** Если ответ не проходит валидацию, скрипт не завершается аварийно. Срабатывает блок `try-except`, который помечает документ флагами `"ai_processed": False` и записывает текст исключения напрямую в MongoDB в поле `"ai_error"` для дальнейшего анализа.

4. Листинг исходного кода

```
import os

import json

import re

from dotenv import load_dotenv

from pymongo import MongoClient

from gigachat import GigaChat

from gigachat.models import Chat, Messages, MessagesRole


load_dotenv()


GIGACHAT_CREDENTIALS = os.getenv("GIGACHAT_CREDENTIALS")

if not GIGACHAT_CREDENTIALS:

    raise ValueError("Ошибка: Не найден GIGACHAT_CREDENTIALS в файле .env")


MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/")

DB_NAME = os.getenv("DB_NAME", "nhl_db")

COLLECTION_NAME = "games"


SYSTEM_PROMPT = """Ты — профессиональный спортивный аналитик и комментатор
NHL. Твоя задача — проанализировать сухую статистику матча и обогатить её
выводами.

Верни СТРОГО валидный JSON-объект. Не пиши сопроводительного текста.
```

ПРАВИЛА:

1. НЕ переводи и не изменяй имена игроков.
2. КРИТИЧЕСКИ ВАЖНО: ЗАПРЕЩЕНО переводить и транслитерировать названия команд на русский язык (не пиши "Чикаго" или "Пингвинз"). Вставляй в русский текст ТОЛЬКО оригинальные английские названия из полей `home_team_name` и `away_team_name`.
3. MVP матча выбирай СТРОГО из авторов голов в разделе `goal_summary`.
4. Выбери от 2 до 5 тегов ТОЛЬКО из этого списка: ["Разгром", "Камбэк", "Сухой матч", "Близкий счет", "Доминанция", "Равная игра", "Волевая победа"].

Структура ответа:

```
{  
  
  "ai_tldr": "Краткая выжимка матча на русском языке, но с командами на английском  
(ПРИМЕР: 'В напряженном матче Chicago Blackhawks одержали победу над Pittsburgh Penguins...')",  
  
  "ai_tags": ["tag1", "tag2"],  
  
  "ai_hidden_insights": {  
  
    "game_intensity": "Low / Medium / High",  

```

```
"mvp_candidate": "Имя лучшего игрока матча",

"turning_point": "Какой момент стал переломным"

}

}

"""

def connect_db():

    client = MongoClient(MONGO_URI)

    return client[DB_NAME]

def build_goal_summary(doc: dict) -> dict:

    goals = doc.get("goals", [])

    scorers_total = {}

    goals_by_period = {}

    for goal in goals:

        scorer_name = goal.get("scorer_name")

        period = goal.get("period")

        time = goal.get("time")

        if not scorer_name: continue

        scorers_total[scorer_name] = scorers_total.get(scorer_name, 0) + 1
```

```
period_key = f"period_{period}"

if period_key not in goals_by_period: goals_by_period[period_key] = []

goals_by_period[period_key].append({"time": time, "scorer_name": scorer_name})

return {"scorers_total": scorers_total, "goals_by_period": goals_by_period}
```

```
def extract_and_parse_json(text_response: str) -> dict:
```

```
    try:
```

```
        match = re.search(r'\{.*\}', text_response, re.DOTALL)
```

```
        if match: return json.loads(match.group(0))
```

```
        return json.loads(text_response)
```

```
    except json.JSONDecodeError:
```

```
        return None
```

```
def validate_schema(data: dict) -> bool:
```

```
    allowed_tags = {"Разгром", "Камбэк", "Сухой матч", "Близкий счет", "Доминация", "Равная игра", "Волевая победа"}
```

```
    if not isinstance(data, dict): return False
```

```
    if not isinstance(data.get("ai_tldr"), str): return False
```

```
    if not isinstance(data.get("ai_tags"), list): return False
```

```
    if not (2 <= len(data["ai_tags"]) <= 5): return False
```

```
    for tag in data["ai_tags"]:
```



```
if tag not in allowed_tags:
```

```
    return False
```

```
insights = data.get("ai_hidden_insights")
```

```
if not isinstance(insights, dict): return False
```

```
if insights.get("game_intensity") not in ["Low", "Medium", "High"]: return False
```

```
if not isinstance(insights.get("mvp_candidate"), str): return False
```

```
if not isinstance(insights.get("turning_point"), str): return False
```

```
return True
```

```
def ask_gigachat(game_data: dict) -> str:
```

```
    data_for_llm = game_data.copy()
```

```
    home_info = data_for_llm.get("home_team_info", [])
```

```
    home_team = home_info[0].get("name", "Unknown Home Team") if home_info else "Unknown Home Team"
```

```
    away_info = data_for_llm.get("away_team_info", [])
```

```
    away_team = away_info[0].get("name", "Unknown Away Team") if away_info else "Unknown Away Team"
```

```
    data_for_llm["home_team_name"] = home_team
```

```
    data_for_llm["away_team_name"] = away_team
```

```
    data_for_llm["goal_summary"] = build_goal_summary(data_for_llm)
```

```
    for key in ['_id', 'home_team_info', 'away_team_info', 'home_team_id', 'away_team_id']:
```

```

data_for_llm.pop(key, None)

user_content = f"Данные матча:\n{json.dumps(data_for_llm, ensure_ascii=False,
default=str)}"

payload = Chat(
    messages=[
        Messages(role=MessagesRole.SYSTEM, content=SYSTEM_PROMPT),
        Messages(role=MessagesRole.USER, content=user_content)
    ],
    temperature=0.1,
    max_tokens=600,
)

with GigaChat(credentials=GIGACHAT_CREDENTIALS, verify_ssl_certs=False) as giga:
    response = giga.chat(payload)

    return response.choices[0].message.content


def main():
    db = connect_db()
    games_col = db[COLLECTION_NAME]

    pipeline = [
        {"$match": {"ai_processed": {"$ne": True}}},
        {"$limit": 15},
        {"$lookup": {"from": "teams", "localField": "home_team_id", "foreignField": "_id", "as":
"home_team_info"}},

```

```

        {"$lookup": {"from": "teams", "localField": "away_team_id", "foreignField": "_id", "as":
"away_team_info"}}
    ]

    games_to_enrich = list(games_col.aggregate(pipeline))

    if not games_to_enrich:

        print("Нет матчей для обогащения (или все успешно обработаны).")

        return

    print(f"Запуск спортивной AI-аналитики (Очередь: {len(games_to_enrich)} матчей)")

    for i, game in enumerate(games_to_enrich, 1):

        game_id = game.get("_id", "Unknown")

        print(f"\n[{i}/{len(games_to_enrich)}] Анализ матча (ID: {game_id})")

        try:

            raw_response = ask_gigachat(game)

            enriched_data = extract_and_parse_json(raw_response)

            if enriched_data and validate_schema(enriched_data):

                games_col.update_one(

                    {"_id": game["_id"]},

                    {"$set": {

                        "ai_tldr": enriched_data.get("ai_tldr"),

                        "ai_tags": enriched_data.get("ai_tags"),

                        "ai_hidden_insights": enriched_data.get("ai_hidden_insights"),

                        "ai_processed": True

                    }},

                    {"$unset": {"ai_error": ""}}

                )

```

```

    )

    tags = ", ".join(enriched_data.get('ai_tags', []))

    print(f"    [+] Успешно! Теги: {tags}")

else:

    raise ValueError("Галлюцинация модели: неверные теги или структура
(отработала валидация)")

except Exception as e:

    print(f"    [-] Ошибка обработки: {e}")

    games_col.update_one(

        {"_id": game["_id"]},

        {"$set": {"ai_processed": False, "ai_error": str(e)}}

    )

)

if __name__ == "__main__":

    main()

```

5. Результаты выполнения скрипта

"C:\PRBD\lab 3-4 PRBD\venv\Scripts\python.exe" "C:\PRBD\lab 3-4 PRBD\nhl_data_enricher.py"

Запуск спортивной AI-аналитики (Очередь: 15 матчей)

[1/15] Анализ матча (ID: 2023020001)

[-] Ошибка обработки: Галлюцинация модели: неверные теги или структура (отработала валидация)

[2/15] Анализ матча (ID: 2023020002)

[+] Успешно! Теги: Камбэк, Равная игра

[3/15] Анализ матча (ID: 2023020003)

[+] Успешно! Теги: Доминация, Равная игра

[4/15] Анализ матча (ID: 2023020004)

[+] Успешно! Теги: Равная игра, Сухой матч

[5/15] Анализ матча (ID: 2023020005)

[+] Успешно! Теги: Камбэк, Волевая победа

[6/15] Анализ матча (ID: 2023020006)

[-] Ошибка обработки: Галлюцинация модели: неверные теги или структура (отработала валидация)

[7/15] Анализ матча (ID: 2023020007)

[+] Успешно! Теги: Доминация, Равная игра

[8/15] Анализ матча (ID: 2023020008)

[+] Успешно! Теги: Доминация, Разгром

[9/15] Анализ матча (ID: 2023020009)

[-] Ошибка обработки: Галлюцинация модели: неверные теги или структура (отработала валидация)

[10/15] Анализ матча (ID: 2023020010)

[+] Успешно! Теги: Доминация, Разгром

[11/15] Анализ матча (ID: 2023020011)

[+] Успешно! Теги: Камбэк, Равная игра

[12/15] Анализ матча (ID: 2023020012)

[+] Успешно! Теги: Равная игра, Близкий счет

[13/15] Анализ матча (ID: 2023020013)

[-] Ошибка обработки: Галлюцинация модели: неверные теги или структура (отработала валидация)

[14/15] Анализ матча (ID: 2023020014)

[+] Успешно! Теги: Доминация, Сухой матч

[15/15] Анализ матча (ID: 2023020015)

[-] Ошибка обработки: Галлюцинация модели: неверные теги или структура (отработала валидация)

Documents 179 Aggregations Schema Indexes 3 Validation

Generate query Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

100 1 - 10 of 10

```
{
  "ai_processed": true
}
```

```
{
  "_id": 2023020002,
  "date": "2023-10-11T00:00:00Z",
  "status": "Final",
  "home_team_id": 5,
  "away_team_id": 16,
  "score": Object,
  "team_stats": Object,
  "goals": Array (6),
  "ai_hidden_insights": Object,
    game_intensity: "High",
    mvp_candidate: "N. Foligno",
    turning_point: "Второй период, когда Chicago Blackhawks забили два гола подряд"
  "ai_processed": true,
  "ai_tags": Array (2),
  "ai_tldr": "В матче Pittsburgh Penguins уступили Chicago Blackhawks со счетом 2:4."
}
```

localhost:27017 > nhl_db > games Open MongoDB shell

Documents 179 Aggregations Schema Indexes 3 Validation

Generate query Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

100 1 - 5 of 5

```
{
  "ai_processed": false
}
```

```
{
  "_id": 2023020001,
  "date": "2023-10-10T21:30:00Z",
  "status": "Final",
  "home_team_id": 14,
  "away_team_id": 18,
  "score": Object,
  "team_stats": Object,
  "goals": Array (8),
  "ai_error": "Галлюцинация модели: неверные теги или структура (отработала валидация_)",
  "ai_processed": false
}
```

```
{
  "_id": 2023020006,
  "date": "2023-10-11T23:30:00Z",
  "status": "Final",
  "home_team_id": 6,
  "away_team_id": 16,
  "score": Object,
  "team_stats": Object,
  "goals": Array (4),
  "ai_error": "Галлюцинация модели: неверные теги или структура (отработала валидация_)",
  "ai_processed": false
}
```

```
{
  "_id": 2023020009,
  "date": "2023-10-12T02:00:00Z"
}
```

6. Вывод

В ходе выполнения второй части лабораторной работы был спроектирован отказоустойчивый конвейер обогащения данных для предметной области NHL. Успешно решена проблема нестабильности генеративных нейросетей: применение агрегаций MongoDB, Data Cleaning и внедрение жесткой системы валидации на основе "белых списков" (Whitelist)

исключило попадание мусорных данных в БД. Скрипт способен восстанавливаться после сбоев, корректно изолирует ошибки валидации и записывает логи прямо в документы коллекции. Поставленные задачи, включая задания на дополнительные баллы (извлечение скрытых инсайтов и обработка LLM-галлюцинаций), выполнены в полном объеме на уровне промышленных стандартов разработки.